EXECUTION BASELINE / VERSION 1.0

Notification Service API
Six-Week Senior Upskill Execution Plan

A repository-aware plan that converts the supplied 6-Week Schedule into sequenced engineering work, measurable quality gates, and review evidence.

PREPARED FOR
Bethoven Acha

PREPARED ON
13 July 2026

TARGET
Grade 4 / Senior readiness

SCHEDULE
6 weeks · approximately 20 hours/week

ANCHOR REPOSITORY
NSA / Notification Service API

BASELINE
main @ 91d94c0 plus current local restructuring

W1–W2
Substantially built; verification and evidence remain

W3–W6
Major implementation phases still planned

5 gates
Two code reviews, presentation, frontend audit, final demo

Planning rule: working code, tests, PRs, ADRs, and recorded demonstrations are the acceptance signal. Course completion is supporting evidence only.

EXECUTIVE DIRECTION

Outcome, boundaries, and planning assumptions

> **Outcome:** evolve the existing .NET 8 API into a demonstrable event-driven system with documented contracts, durable delivery, frontend rendering examples, repeatable infrastructure, and a tested blue/green release path.

What this plan covers

- Every activity and deliverable named in the supplied 6-Week Schedule.
- Repository gaps discovered in the 13 July 2026 local audit.
- Daily engineering tasks sized to roughly four focused hours.
- Acceptance checks, evidence locations, review gates, dependencies, and risks.

What requires human coordination

- EM confirms Week 1 start date, code reviews, presentation slot, and Grade 4 rubric.
- Bethoven takes and submits Skill IQ assessments.
- EM supplies the ten React hook snippets.
- GitHub, Pluralsight, GHCR, Docker Desktop, and presentation access are available.

Operating model

Cadence	Execution rule
Start of week	Confirm prerequisites, branch from a green main branch, and lock the week's Definition of Done.
Each day	Work in small commits; update evidence; send EM **Done · Blocked · Tomorrow**.
Before merge	Build, tests, security/configuration checks, documentation, and demo script must pass.
End of week	Tag the baseline, record the demo, capture review findings, and carry only explicit remediation forward.
Change control	If a requirement changes, update this plan and an ADR before changing architecture.

Status language used in this plan

EVIDENCED

Present in the local repository and buildable.

VERIFY / HARDEN

Implemented, but acceptance evidence or edge-case tests are missing.

PLANNED

No repository evidence was found; implementation is scheduled.

EXTERNAL

Requires an account, EM input, live review, or assessment.

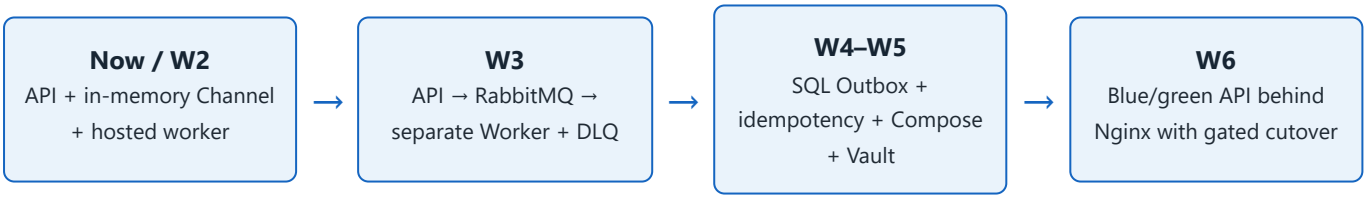
> **Baseline caution:** the local worktree contains extensive uncommitted folder restructuring. The first execution action is to review and preserve those changes in a clean baseline commit or PR; no reset or bulk cleanup should be performed.

REPOSITORY-AWARE GAP ANALYSIS

Current state against the six-week schedule

Week	State	Evidence already present	Work still required
1	VERIFY / HARDEN	.NET 8 CRUD API; Swagger; XML docs; EF migrations; draw.io architecture file; project builds cleanly.	Skill IQ proof; GitHub/PR proof; documentation completeness audit; 30-minute design evidence; one-page decision record; repeatable smoke checks.
2	VERIFY / HARDEN	v1/v2 routes; deprecation/sunset headers; Problem Details; 3 ADRs; Polly retry/circuit breaker; in-memory bulk queue, worker, and status endpoint.	Automated error/version/resilience tests; measured 202 latency under 100 ms; ADR alternatives/tradeoffs; enabled-provider failure demo; EM Review #1.
3	PLANNED	Week 2 in-process flow provides a migration starting point.	RabbitMQ; separate WorkerService; persisted job status; DLQ; Docker Desktop run; 10 React fixes; ADR presentation.
4	PLANNED	EF Core and SQL Server already in use.	Idempotency; transactional Outbox; broker-down recovery proof; four-mode Next.js app; CI and GHCR workflow; EM Review #2.
5	PLANNED	Current application has a SQL dependency and clear configuration points.	React polish; Angular OnPush demo; full Compose stack; Vault bootstrap; re-runnable provisioning; unit/integration test expansion.
6	PLANNED	No deployment automation found.	Nginx blue/green topology; health-gated switch; one-command rollback; availability proof; retakes; final live assessment.

Architecture evolution



Proposed repository shape by Week 5

Area	Purpose
`src/NSA.Api`	HTTP contracts, OpenAPI, versioning, job submission and status.
`src/NSA.Worker`	RabbitMQ consumer, retry/dead-letter handling, notification delivery.
`src/NSA.Domain · Application · Infrastructure`	Shared domain rules, use cases/contracts, persistence and provider adapters. Split only when it reduces coupling; avoid a cosmetic rewrite.
`tests/NSA.UnitTests · NSA.IntegrationTests`	Fast domain/application tests and container-backed API/broker/database tests.
`web/rendering-lab · web/angular-onpush-demo`	Next.js rendering-mode evidence and focused Angular change-detection evidence.
`deploy/compose · deploy/blue-green`	Local infrastructure, Vault bootstrap, Nginx switching, rollback and demo scripts.
`docs/adr · evidence/week-XX`	Decisions and reviewable proof: commands, screenshots, timings, and demo notes.

The exact project split is confirmed in the Week 3 topology ADR before files move. The schedule requires an API and separate WorkerService; additional libraries are optional and must earn their complexity.

WEEK 1

OPENAPI FOUNDATION + ARCHITECTURE REFRESH

Stabilize and prove the existing foundation

API documentation · System design · Baseline assessments

> **Week objective:** convert the current implementation into a clean, reviewable Week 1 baseline rather than rebuilding working features. Record external assessments before course work.

Current evidence

.NET 8 API, CRUD controllers, EF Core, Swagger at `/swagger`, XML endpoint comments, acceptance notes, and `drawio/notification.drawio`.

Primary gaps

Skill IQ evidence, GitHub/PR evidence, documentation completeness, decision-paper linkage, timed design proof, and automated smoke coverage.

Five-day execution sequence

Day 1	**W1-01** Take C#, ASP.NET Core, and React Skill IQ baselines before courses; send results to EM. **W1-02** Review the dirty worktree, build, run migrations, and establish a protected baseline branch/PR.	**4 h**
Day 2	**W1-03** Inventory every CRUD route, request/response, validation rule, and status code. Complete missing XML `summary` / `response` documentation and OpenAPI response metadata.	**4 h**
Day 3	**W1-04** Add repeatable build/start/Swagger checks and CRUD smoke scenarios. Verify OpenAPI JSON is valid and Swagger shows schemas, versions, success codes, and failure codes.	**4 h**
Day 4	**W1-05** Run the 30-minute order-tracking whiteboard exercise. Export the draw.io diagram to PDF/PNG. Create a one-page decision note covering context, constraints, design, alternatives, and tradeoffs.	**4 h**
Day 5	**W1-06** Rewrite README setup/demo instructions, publish the GitHub PR, run the Week 1 demo script, resolve critical findings, and tag the accepted baseline.	**4 h**

Exit criteria

- ☐ All three baseline scores are timestamped and sent before courses.
- ☐ `dotnet build` is clean and application startup is repeatable.
- ☐ Notification CRUD and supporting routes pass smoke checks.
- ☐ `/swagger` loads and OpenAPI documents all endpoint outcomes.
- ☐ Architecture source and exported diagram are readable.
- ☐ One-page decision document explains alternatives and tradeoffs.
- ☐ GitHub PR/repository link and demo evidence are captured.
- ☐ README enables a reviewer to run the solution without oral help.

Required evidence

```
evidence/week-01/{skill-iq-record.md, build.txt, swagger.png, crud-smoke.md, architecture-export.pdf, demo-notes.md}
```

> **Decision needed:** confirm with the EM whether the Week 1 one-page decision note can become ADR 001 or must remain a separate artifact.

WEEK 2

ERROR DESIGN, VERSIONING + ASYNC FOUNDATIONS

Test and demonstrate the features already built

Problem Details · API versions · ADRs · Polly · Background jobs

Week objective: harden the existing Week 2 implementation with explicit contracts and measurable proof, then pass EM Code Review #1.

Current evidence

v1/v2 routes, deprecation headers, Problem Details handler/pages, three ADRs, typed Postbound client with retry/circuit breaker, in-memory job channel, hosted worker, 202 and status routes.

Primary gaps

No automated tests, resilience transcript, latency measurement, or external review. Polly is bypassed while Postbound is disabled; bulk work currently persists records but does not call the sender.

Five-day execution sequence

Day 1	**W2-01** Define and automate the error matrix: model validation, domain validation, missing resource, unsupported version, malformed request, method mismatch, and unexpected exception. Require RFC 7807 media type and trace ID.	**4 h**
Day 2	**W2-02** Test v1/v2 routes and headers. Refine all three ADRs to record status, context, decision drivers, considered options, decision, consequences, and validation method.	**4 h**
Day 3	**W2-03** Create a deterministic failing HTTP test server/handler. Prove three exponential retries and circuit opening; document how the provider prevents a retried email POST from duplicating delivery.	**4 h**
Day 4	**W2-04** Confirm whether "bulk notification" means record creation or delivery, then test that contract, job states, and failure counts. Submit at least 20 requests and record p95; require HTTP 202 plus job ID with p95 <100 ms locally.	**4 h**
Day 5	**W2-05** Run EM Code Review #1 using a scripted demo. Address blocking findings, record non-blocking debt, merge, and tag `week-2-complete`.	**4 h**

Exit criteria

- ☐ v1 and v2 are callable; v1 responses include Deprecation and Sunset.
- ☐ Every enumerated error returns consistent Problem Details.
- ☐ Three accepted ADRs include alternatives and tradeoffs.
- ☐ Retry delays and circuit-open behavior are demonstrated by tests/logs.
- ☐ Bulk POST returns 202 and a stable status URL/job ID.
- ☐ Status visibly transitions Queued → Processing → terminal state.
- ☐ Recorded p95 response time is below 100 ms under stated conditions.
- ☐ EM Review #1 findings and disposition are stored.

Required evidence

```
evidence/week-02/{error-matrix.md, versioning.http, polly-test.txt, bulk-latency.json, job-lifecycle.md, em-review-01.md}
```

Scope boundary: Week 2 intentionally uses an in-memory queue. Restart durability and broker failure recovery are Week 3–4 outcomes.

WEEK 3

MESSAGE QUEUES + ARCHITECTURE PRESENTATION

Move asynchronous work out of process

RabbitMQ · WorkerService · DLQ · React hooks · Communication

> **Week objective:** replace the in-memory worker with a Docker-hosted RabbitMQ flow and a separate WorkerService while keeping job progress observable across processes.

Target message flow

API
Create persisted job; publish versioned command

→

RabbitMQ
Main exchange/queue; durable message

→

Worker
Manual ack; update SQL progress

→

DLQ
Poison message after bounded attempts

Five-day execution sequence

Day 1	**W3-01** Write/approve the queue-topology ADR: exchange, queue, routing keys, durable message schema/version, manual acknowledgements, retry limit, poison classification, DLX/DLQ names, correlation IDs, and observability.	**4 h**
Day 2	**W3-02** Add RabbitMQ publisher and a separate .NET WorkerService. Persist job/progress in SQL so API and worker share state. Keep provider delivery behind `IEmailSender`.	**4 h**
Day 3	**W3-03** Add Dockerfiles and a Week 3 Compose stack for API, worker, RabbitMQ Management, and SQL Server. Add readiness waits, health checks, named volumes, and non-secret local defaults.	**4 h**
Day 4	**W3-04** Test happy path, worker restart/redelivery, broker restart, malformed message, poison delivery, DLQ inspection, and correlation. Capture the RabbitMQ UI DLQ proof.	**4 h**
Day 5	**W3-05** Correct the ten EM-supplied React hook snippets with explanations. Present the queue ADR for 15 minutes: context, options, tradeoffs, decision, failure demo, and questions.	**4 h**

Exit criteria

- ☐ API and WorkerService are independently deployable processes.
- ☐ Entire Week 3 stack starts on Docker Desktop.
- ☐ Successful messages are acknowledged only after durable state change.
- ☐ Poison message reaches named DLQ after bounded attempts.
- ☐ Status remains queryable from the API while worker processes.
- ☐ Ten hook fixes cover dependencies, cleanup, and stale closures.
- ☐ Gist link and explanations are submitted.
- ☐ Architecture presentation is delivered and feedback recorded.

Required evidence

```
evidence/week-03/{topology.md, compose-start.txt, happy-path.md, dlq.png, restart-redelivery.md, react-gist-link.md, presentation.pdf, feedback.md}
```

> **Design warning:** RabbitMQ acknowledgement gives at-least-once delivery, not exactly once. Week 4 adds idempotency/inbox safeguards and transactional publication.

WEEK 4

IDEMPOTENCY / OUTBOX + NEXT.JS + GITHUB ACTIONS

Make delivery durable and automate validation

EF Core Outbox · Idempotency · Rendering models · CI/CD · GHCR

> **Week objective:** ensure an accepted command survives broker downtime and duplicate client/queue delivery, then publish demonstrable backend and frontend artifacts through a passing pipeline.

Correctness contract

Concern	Planned behavior	Acceptance proof
Client duplicate	Scope `X-Idempotency-Key` to event-producing POSTs, starting with bulk notifications. Same key + same request replays the original response/job.	Two sequential and two concurrent submissions create one job and return the same result.
Key misuse	Store request hash with key; same key + different body returns HTTP 409 Problem Details.	Postman/automated conflict scenario.
Broker unavailable	Store business change and Outbox row in one SQL transaction. Relay retries unpublished rows with backoff and publisher confirms.	Commit while broker is down; start broker; reconcile one published event and processed job.
Consumer duplicate	Use stable message ID and a unique processed-message/inbox record in the worker transaction.	Redeliver the same message; side effect occurs once.

Five-day execution sequence

Day 1 **W4-01** Approve idempotency scope, key/hash/expiry schema, concurrency behavior, response replay, conflict response, and cleanup policy. Add migration and integration tests first. **4 h**

Day 2 **W4-02** Add Outbox and worker Inbox records, transactional writes, relay leasing, retry metadata, publisher confirms, correlation, and safe cleanup. Document at-least-once semantics. **4 h**

Day 3 **W4-03** Run broker-down, process-crash, concurrent-key, and duplicate-delivery tests. Reconcile database rows, RabbitMQ messages, job status, and notification side effects. **4 h**

Day 4 **W4-04** Create the Next.js rendering lab: SSG, SSR, ISR, and CSR pages with visible render/fetch timestamps and concise explanations of cache/revalidation behavior. **4 h**

Day 5 **W4-05** Create GitHub Actions from scratch: PR = restore/build/test only; main/tag = test, publish, containerize, and push immutable image to GHCR. Run EM Code Review #2. **4 h**

Exit criteria and evidence

- ☐ Idempotency replay, conflict, and concurrency checks pass.
- ☐ No accepted event is lost during demonstrated broker downtime.
- ☐ Duplicate delivery does not duplicate business side effects.
- ☐ Four rendering pages show correct, visible timestamps.
- ☐ PR workflow never publishes an image.
- ☐ Main/tag workflow pushes a commit-SHA image to GHCR.
- ☐ Queue and DLQ remain demonstrable in Review #2.
- ☐ Review findings and follow-up owners are recorded.

```
evidence/week-04/{idempotency.postman.json, broker-down.md, outbox-reconciliation.sql, duplicate-delivery.md,
rendering-modes.md, actions-run.md, ghcr-image.txt, em-review-02.md}
```

> **Sequencing correction:** the original schedule places test backfill in Week 5, but Week 4 CI requires tests. Add the minimum critical unit/integration suite in Weeks 2–4; Week 5 expands coverage and balances the testing pyramid.

WEEK 5

FRONTEND POLISH + COMPOSE AND VAULT PROVISIONING

Make the full stack repeatable on a clean machine

React hooks · Angular OnPush · Docker Compose · Vault · xUnit

> **Week objective:** close the frontend precision gap and provide an idempotent local provisioning path in which application secrets come from Vault rather than tracked configuration.

Five-day execution sequence

Day 1	**W5-01** Audit the Next.js app for effect dependencies, cleanup, cancellation, stale closures, and unnecessary effects. Add focused React tests and document each correction.	**4 h**
Day 2	**W5-02** Build a small Angular OnPush demo showing immutable vs mutated input, `async` pipe/observable update, signals, and explicit change marking. Prepare a five-minute explanation.	**4 h**
Day 3	**W5-03** Consolidate Compose services: network, volumes, SQL Server, RabbitMQ, Vault, API, worker, and frontends as needed. Add health checks and dependency readiness.	**4 h**
Day 4	**W5-04** Create re-runnable `provision.sh`: initialize/unseal Vault if needed, enable a secret engine, seed generated local secrets, configure least-privilege app auth, migrate database, and start/verify services.	**4 h**
Day 5	**W5-05** Expand xUnit unit and container-backed integration tests; document the testing pyramid. Rehearse twice from a clean Docker state and complete the frontend audit checkpoint.	**4 h**

Vault and secret-handling guardrails

Do

- Generate bootstrap material locally.
- Store init/unseal and app credentials only in ignored local files or Docker secrets.
- Give API/worker read access only to their paths.
- Redact logs and evidence.

Do not

- Commit root tokens, unseal keys, passwords, or API keys.
- Use a fixed demo token in Compose.
- Print secrets in CI or provisioning output.
- Call a successful start “ready” without health checks.

Exit criteria

- ☐ React audit has fixes, explanations, and passing tests.
- ☐ Angular demo visibly proves OnPush behavior.
- ☐ Compose includes required network, volumes, RabbitMQ, SQL, and Vault.
- ☐ No runtime secret is hardcoded in tracked files.
- ☐ `provision.sh` succeeds twice without manual repair.
- ☐ Clean-machine prerequisites and Windows shell choice are documented.
- ☐ Unit and integration suites run locally and in CI.
- ☐ Frontend audit feedback is captured and resolved/assigned.

Required evidence

```
evidence/week-05/{react-audit.md, angular-demo.mp4-or-link, compose-config.txt, vault-policy.hcl, secret-scan.txt,
provision-run-1.txt, provision-run-2.txt, test-report.xml, frontend-audit.md}
```

> **Environment decision:** because the developer machine is Windows and the requirement names `provision.sh`, document Git Bash or WSL as a prerequisite and keep the script POSIX-compatible.

WEEK 6

ZERO-DOWNTIME DEPLOY + FINAL READINESS

Prove promotion, rollback, and senior-level ownership

Nginx · Blue/green · Health gates · Pipeline demo · Final assessment

Week objective: automate a health-gated deployment to the idle color, switch traffic without observed request failure, and restore the previous color with one command.

Release sequence

Build Immutable SHA-tagged image → **Deploy idle** Blue or green; migrate safely → **Gate** Readiness + smoke checks → **Switch** Nginx reload; observe; rollback ready

Five-day execution sequence

Mon	**W6-01** Retake C#, ASP.NET Core, and React Skill IQ assessments; compare to Week 1 and send results. Freeze feature scope and confirm Friday assessment/demo rubric.	**4 h**
Tue	**W6-02** Add liveness/readiness endpoints. Configure Nginx, blue/green API services, active-color state, immutable image selection, and backward-compatible database migration rules.	**4 h**
Wed	**W6-03** Implement deploy-idle → health/smoke gate → Nginx switch → observation workflow. Abort automatically when the idle color is unhealthy.	**4 h**
Thu	**W6-04** Implement one-command rollback. Run continuous traffic through successful cutover, failed-health promotion, and rollback; collect availability and recovery results.	**4 h**
Fri	**W6-05** Live walkthrough: architecture, ADRs, API contracts, DLQ, Outbox recovery, frontends, Vault provisioning, CI image, blue→green switch, and rollback. Complete Grade 4 decision.	**4 h**

Measurable demonstration contract

Scenario	Pass condition
Normal cutover	At least 1,000 requests at a documented steady rate across the switch, with zero network/non-2xx failures attributable to deployment.
Unhealthy idle color	Readiness/smoke gate fails; active upstream remains unchanged; pipeline reports a clear failure.
Rollback	One documented command restores the previous healthy color in ≤60 seconds, with the same traffic probe running.
Data compatibility	Both colors can operate during the switch; migration does not require an immediate destructive schema change.

Exit criteria and evidence

- ☐ Retake scores and comparison are submitted.
- ☐ Pipeline always deploys the idle color.
- ☐ Unhealthy release cannot receive production traffic.
- ☐ Cutover passes the stated availability probe.
- ☐ Rollback is one command and meets the target.
- ☐ Provisioning and pipeline demos are repeatable.
- ☐ Final walkthrough covers all six-week artifacts.
- ☐ Grade 4 go/no-go decision and actions are recorded.

`evidence/week-06/{skill-iq-comparison.md, pipeline-run.md, cutover-results.json, failed-health.md, rollback-results.json, final-demo-script.md, readiness-assessment.md}`

QUALITY MANAGEMENT

Definition of Done and verification strategy

> **Release rule:** a feature is not done when it merely runs once. It is done when its contract is automated where practical, its failure path is demonstrated, its operation is documented, and its evidence is reviewable.

Definition of Done for every engineering item

- ☐ Requirement and acceptance condition are linked in the PR.
- ☐ Code builds with zero warnings introduced by the change.
- ☐ Unit/integration/contract tests appropriate to the risk pass.
- ☐ Negative and recovery paths are tested, not only happy path.
- ☐ Public API/OpenAPI and configuration docs are current.
- ☐ No secret or personal token is tracked or printed.
- ☐ Structured logs include correlation/job/message identifiers.
- ☐ Migration and rollback/recovery implications are documented.
- ☐ Evidence is saved under the correct week directory.
- ☐ Review findings are fixed or assigned with owner/date.

Test pyramid target by Week 5

Layer	Purpose	Priority scenarios	Execution
Unit largest set	Domain and deterministic application behavior	Validation, status transitions, request hashing, idempotency decisions, Outbox state, retry classification.	Every build/PR
Integration focused set	Database, HTTP pipeline, broker, Vault boundaries	Problem Details, SQL transactions, Outbox relay, Inbox dedupe, Rabbit ack/DLQ, migrations, secret retrieval.	PR and main
End-to-end small critical set	Cross-service business and operational flows	Bulk submit-to-completion, broker outage recovery, Compose provisioning, blue/green cutover and rollback.	Main/nightly/demo
Manual evidence	UI, communication, assessment	Swagger review, RabbitMQ DLQ UI, rendering timestamps, React Gist, Angular demo, architecture presentation, EM gates.	Milestones

Milestone gates

Gate	Must be demonstrable	Approver / record
W2 Review #1	Documented API, v1/v2, all error shapes, 202 lifecycle, Polly failure behavior, three ADRs.	EM · `em-review-01.md`
W3 Presentation	15-minute ADR story with live queue/DLQ and explicit tradeoffs.	EM/team · feedback record
W4 Review #2	Idempotency, Outbox recovery, queue/DLQ, rendering lab, tests, passing CI and GHCR image.	EM · `em-review-02.md`
W5 Frontend/infra	Hook precision, OnPush, re-runnable provisioning, Vault-secret proof, test pyramid.	EM/frontend reviewer
W6 Readiness	Pipeline, health gate, cutover, rollback, full architecture ownership and assessment results.	EM · go/no-go record

DELIVERY RISK MANAGEMENT

Dependencies, risks, and mitigations

Risk / dependency	Level	Mitigation and trigger	Owner / due
Start date and EM gates are not scheduled.	HIGH	Book both reviews, W3 presentation, W5 audit, and W6 Friday assessment before Week 1. Escalate if any gate has no slot by W1 Day 2.	EM + Bethoven Before W1
Ten React snippets and Grade 4 rubric are external.	HIGH	Request snippets by W2 and rubric by W4. If unavailable, use an agreed substitute set and document the deviation.	EM W2 / W4
Approximately 120 hours for a broad full-stack program.	MEDIUM	Timebox courses, reuse the anchor project, enforce weekly scope, and prioritize acceptance paths. Carry work only through an explicit review decision.	Bethoven Daily
Current worktree has many uncommitted moves.	HIGH	Review, build, and checkpoint the local state before architectural splitting. Never clean/reset files without confirming ownership.	Bethoven W1 Day 1
No automated test projects currently found.	HIGH	Add critical tests in Weeks 1–4 so CI has value; expand in Week 5. Do not postpone all test work to Week 5.	Bethoven Start W1
Retries or at-least-once delivery can duplicate email.	MEDIUM	Document at-least-once Rabbit delivery. Use stable provider/message IDs plus Inbox/idempotent side effects; do not enable unsafe POST retries without a provider deduplication contract.	Bethoven W2–W4
Vault bootstrap can leak root material.	HIGH	Generate locally, store only in ignored/Docker-secret locations, redact output, use least-privilege app auth, and run a secret scan.	Bethoven W5
`provision.sh` on Windows is underspecified.	MEDIUM	Standardize on Git Bash or WSL, validate line endings and POSIX syntax, list prerequisites, and test on a clean profile.	Bethoven W5
Blue/green with shared database can break old color.	HIGH	Use expand/contract migrations; keep both versions compatible during cutover; block promotion on migration/readiness failures.	Bethoven W6
GHCR/pipeline credentials or permissions unavailable.	MEDIUM	Verify package permissions by W3; use GitHub-provided token with least privilege; never use a personal token unless approved.	Bethoven + EM W3

Decisions to confirm before execution

1. Calendar start date, five-day working pattern, reviewers, and demo slots.
2. Whether Week 1's decision page is separate from ADR 001.
3. Exact v1/v2 semantic difference, if the EM expects more than parallel routes.
4. RabbitMQ client/library and topology; job-retention period; retry and DLQ thresholds.
5. Idempotency endpoint scope and retention; Next.js/router version; GitHub branch and image conventions.
6. Clean-machine definition, Git Bash versus WSL, Vault mode, and final readiness rubric.

> **Stop-the-line criteria:** exposed secret, data-loss evidence, destructive migration incompatible with the active color, failing main build, or a health gate that can promote an unhealthy release. These must be corrected before proceeding.

EXECUTION CONTROL

How this PDF becomes the working baseline

Kickoff checklist

- ☐ EM confirms start date, reviewers, external inputs, and Grade 4 rubric.
- ☐ Engineer records environment versions and validates required accounts/tools.
- ☐ Current local changes are reviewed and checkpointed safely.
- ☐ Week 1 branch/PR and evidence directory are created.
- ☐ Skill IQ baseline timing is confirmed before course access.
- ☐ Daily EOD message time and channel are agreed.
- ☐ Risk owners accept the register on page 11.
- ☐ Any approved deviation is written into the plan/ADR.

Weekly control record

Week	Planned outcome	Gate date	Result
1	Documented API foundation + architecture decision	_____	☐ Pass ☐ Carry ☐ Replan
2	Error/version/async proof + EM Review #1	_____	☐ Pass ☐ Carry ☐ Replan
3	RabbitMQ/DLQ + hooks Gist + presentation	_____	☐ Pass ☐ Carry ☐ Replan
4	Idempotency/Outbox + Next.js + CI + Review #2	_____	☐ Pass ☐ Carry ☐ Replan
5	Frontend audit + Compose/Vault provisioning	_____	☐ Pass ☐ Carry ☐ Replan
6	Blue/green demo + Grade 4 decision	_____	☐ Pass ☐ Carry ☐ Replan

Daily EOD template

Done: task IDs completed, PR/commit, tests run, evidence path.

Blocked: exact blocker, impact, checks already tried, owner/decision needed.

Tomorrow: next task IDs, planned validation, review request.

Risk change: new/escalated/closed risk and mitigation.

Final evidence index

Evidence family	Expected final artifacts
Architecture	Diagram/export, ADR set, presentation deck, review feedback.
API and resilience	OpenAPI, error matrix, version checks, Polly transcript, latency and job lifecycle results.
Messaging correctness	Topology, DLQ proof, Outbox reconciliation, broker outage and duplicate-delivery results.
Frontend	React Gist, rendering-mode lab, hook audit, Angular OnPush demo.
Delivery and security	CI runs, GHCR image, Compose/Vault proof, secret scan, provisioning transcripts.
Operations/readiness	Health-gate failure, cutover/rollback metrics, demo script, assessment and go/no-go record.

Plan completion condition: all six weekly gates are passed or have explicitly accepted remediation, all required external events are recorded, and the final readiness decision is signed off by the EM.